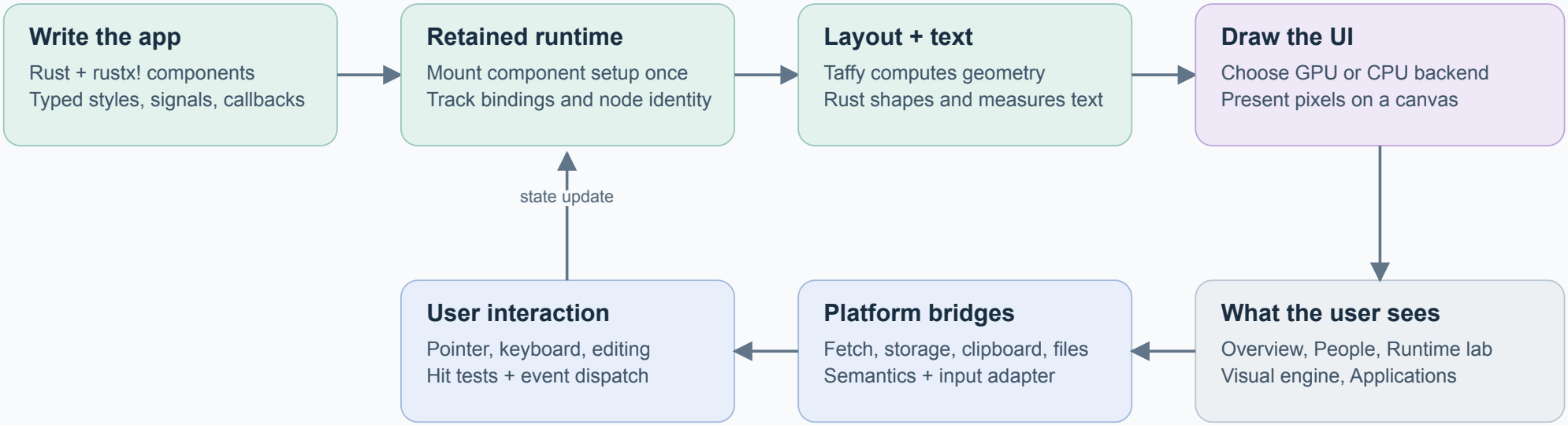
One app, owned by Rust

Read the arrows from authoring to pixels. The browser supplies platform capabilities around the Rust UI engine.



COLOR KEY — Rust ownership — Browser / platform — GPU rendering

Visible UI

Rust owns component state, geometry, shaped text, editing and drawing. Canvas displays the result.

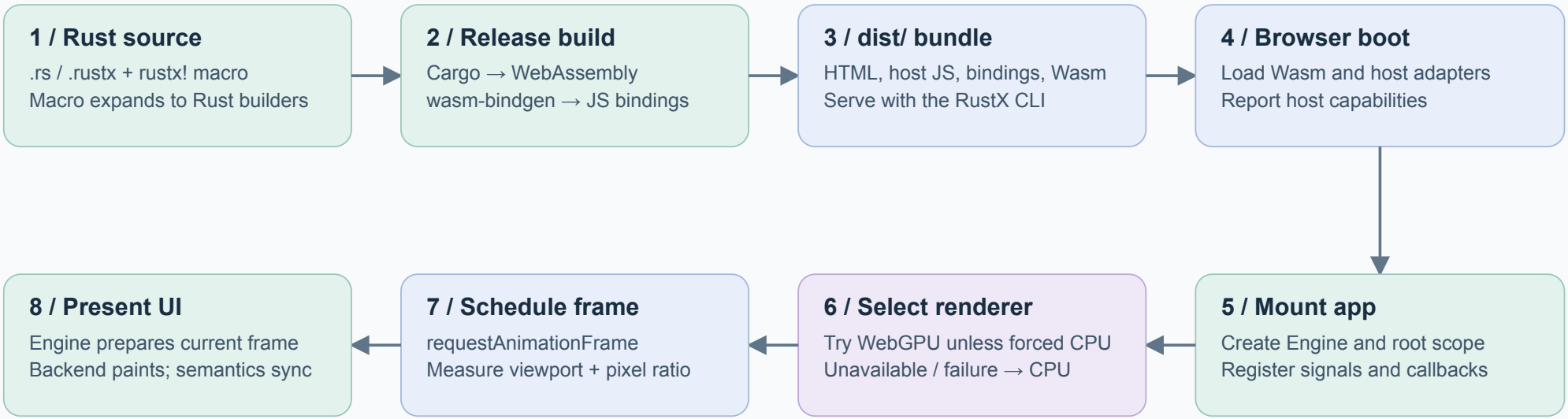
Browser support

A semantic DOM projection and input/textarea bridge provide accessibility and platform text input. They do not compute visible component layout.

Sources: docs/BLUEPRINT.md §4–5 · examples/lab/src/app.rustx · crates/rustx-engine/src/lib.rs

From source code to the first frame

Build time creates the runnable browser bundle. Startup then mounts the application and selects a renderer.



NATIVE PATH The same app + engine compile to a native binary; winit and native adapters replace the browser host.

Mount once

Component setup runs when its instance mounts. Signal changes update reactive bindings. A route or keyed remount may create a new instance.

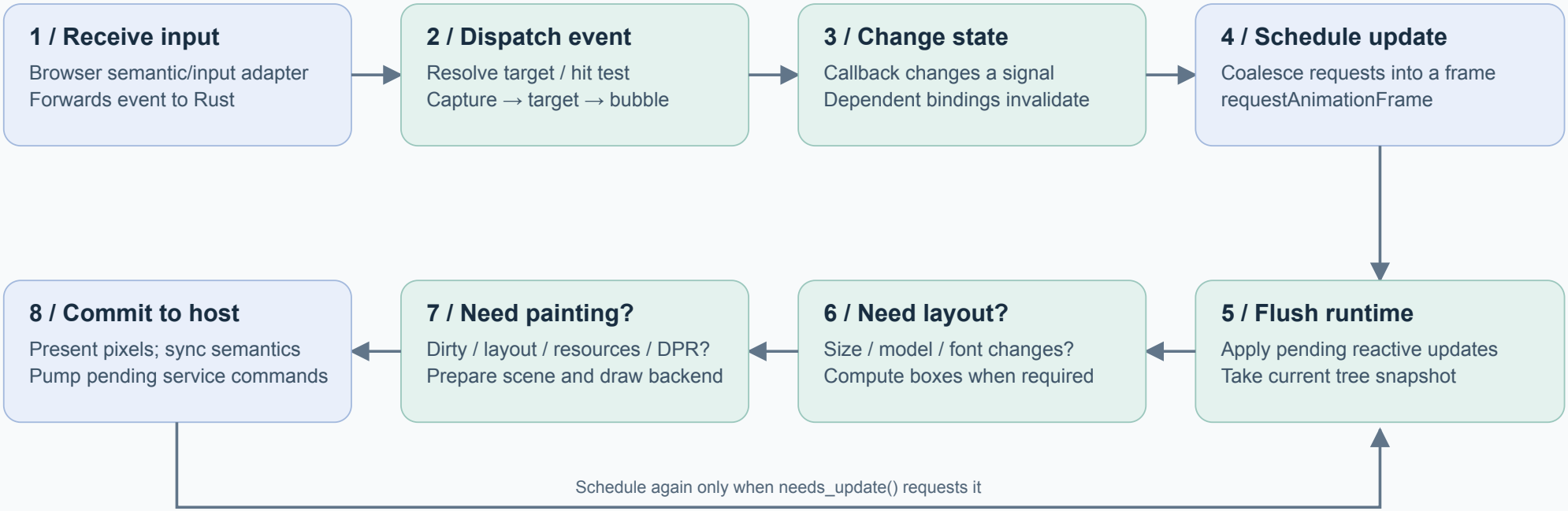
Renderer recovery

Browser device loss or GPU drawing failure switches to CPU while keeping the application engine. An explicit retry can initialize GPU again.

Sources: scripts/build-web.sh · web/host.js · examples/lab/src/lib.rs · crates/rustx-engine/src/host.rs

What happens when you click?

Example: increasing a counter. The same loop also handles scrolling, editing and asynchronous responses.



When idle

If nothing requires repainting, Engine::frame returns false. The browser skips presentation and does not continuously redraw a settled app.

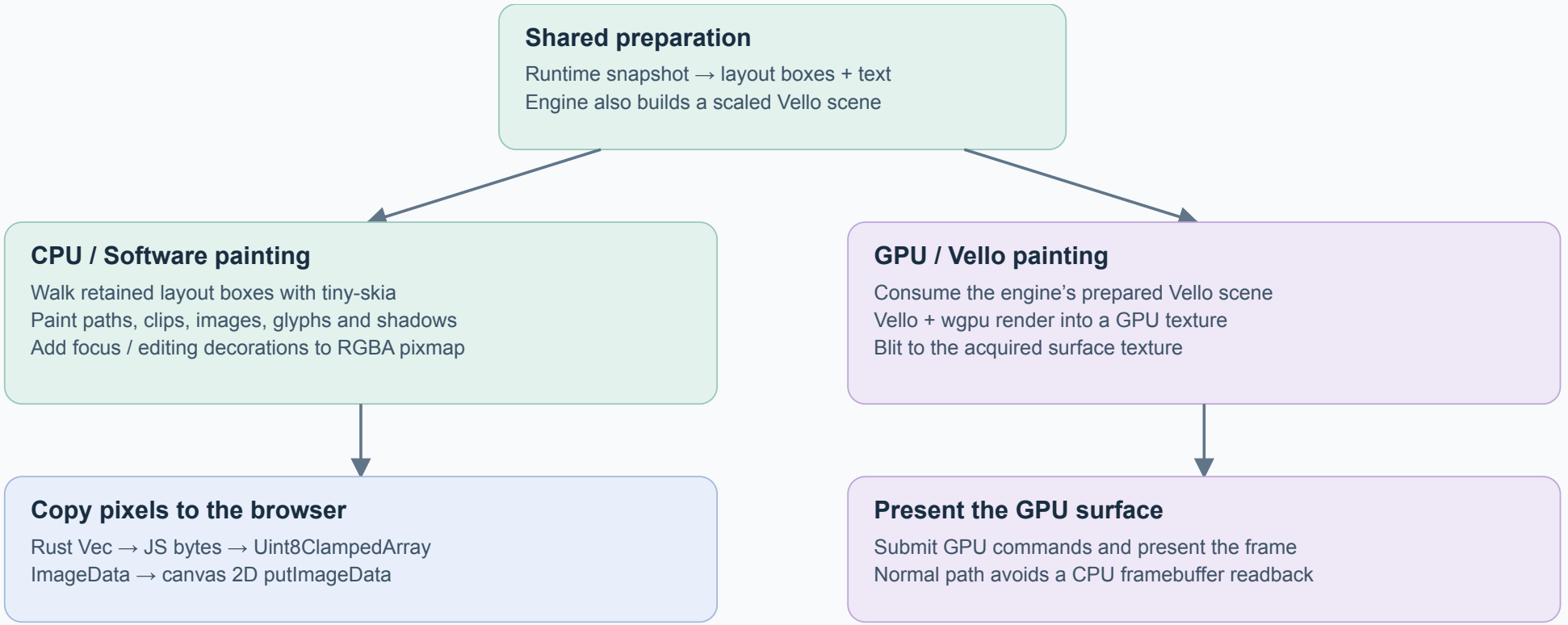
Important distinction

Retained layout and cached scene fragments reduce preparation work. In the CPU path, a changed frame still clears and repaints the pixel buffer.

Sources: web/host.js draw()/schedule() · crates/rustx-engine/src/lib.rs Engine::frame · rustx-core runtime

How Rust turns UI into pixels

Both paths share Rust layout and shaped text. They perform the final painting differently.



CPU is a separate painter

It paints from layout boxes, not by replaying Vello's GPU commands. Shared state and text positions do not imply identical rasterization.

What is preserved

Both routes keep layout and text in Rust. Native presentation uses native surfaces instead of browser ImageData / canvas APIs.

Sources: rustx-web/src/lib.rs WebApp::frame / WebGpu::draw · rustx-engine/src/cpu.rs · examples/lab/src/lib.rs pixels()

Why CPU mode feels slower

These are implementation costs visible in source. Their relative impact needs measurement on the actual device.

A / More pixel work

Changed frame clears full framebuffer
Paths and blending execute in software
DPR 2 = 4× the pixels of DPR 1

B / Repeated temporary buffers

Full-canvas clip masks for clipped nodes
A colored pixmap per painted glyph
Extra layers for group opacity

C / Software effects + copies

Two-pass shadow blur with nested loops
Pixel Vec copied across Wasm / JS
New typed array + putImageData

CPU work occupies the UI thread

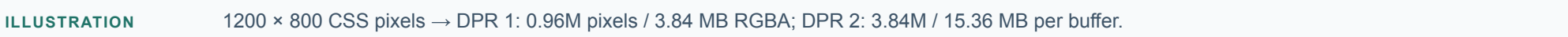
Browser input waits for synchronous work
Long frames feel like scroll / typing lag

GPU performs parallel raster work

Vello submits work to the GPU
Hardware draws and blends many pixels

Potential improvements

Partial repaint; smaller / reused masks
Cached text bitmaps; fewer frame copies



Existing optimizations

Demand-driven scheduling, text/glyph caches, retained layout, scene-fragment reuse and drawing culling are already present.

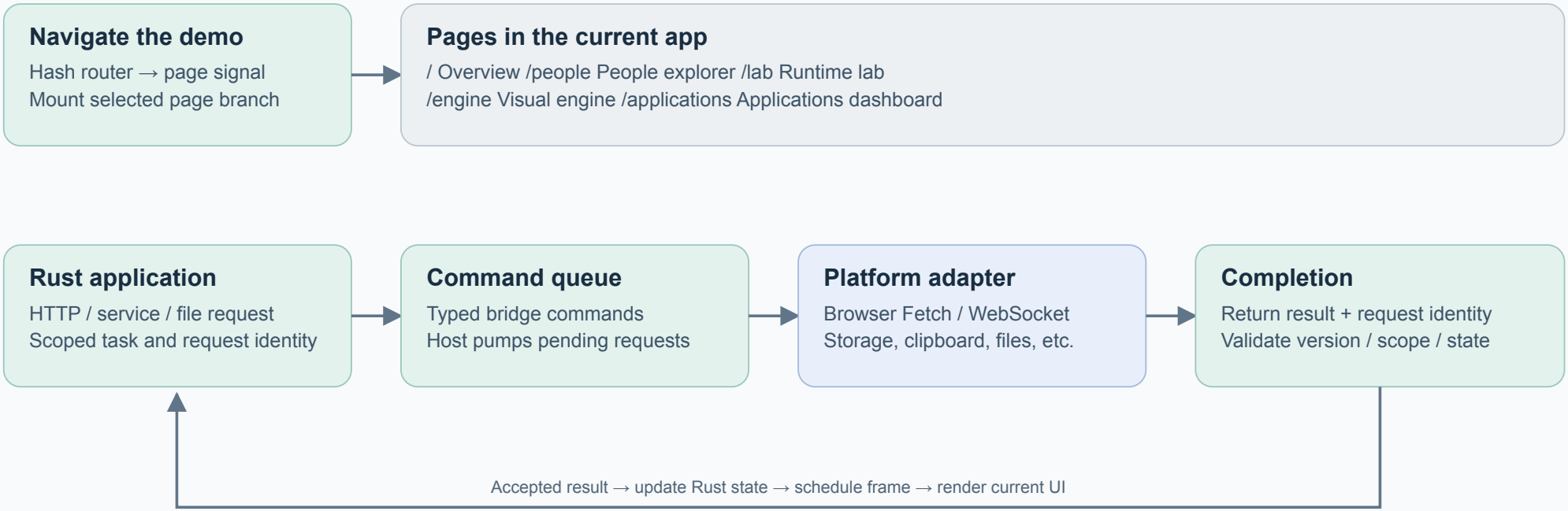
No invented benchmark

This diagram explains possible costs. It does not establish an FPS, a CPU/GPU speed ratio, or which operation dominates your device.

Sources: rustx-engine/src/cpu.rs draw / paint_node / paint_shadow / paint_text · web/host.js · examples/lab/src/lib.rs

Navigation, data and platform services

The demo’s pages share the same engine. External work returns through adapters and re-enters the update loop.



Semantic tree → browser accessibility DOM / native AccessKit. Input bridge → Rust text model → painted caret and selection.

Read the implementation

Start with app.rustx → examples/lab/src/lib.rs → web/host.js → Engine::frame → scene.rs / cpu.rs → rustx-web GPU module. Local paths are relative to the project root.

Scope of this guide

Source-based explanation of the current Stage 3 implementation. It adds no Windows, real OS IME, screen-reader or performance acceptance claims; see docs/ACCEPTANCE.md.

Sources: examples/lab/src/app.rustx · web/services.js · web/semantics.js · docs/STAGE3_SERVICES.md · docs/ACCEPTANCE.md